

Manual Técnico

UNO y UNO Flip



Introducción

Juego escrito en C++ basado en el juego de cartas UNO y UNO flip

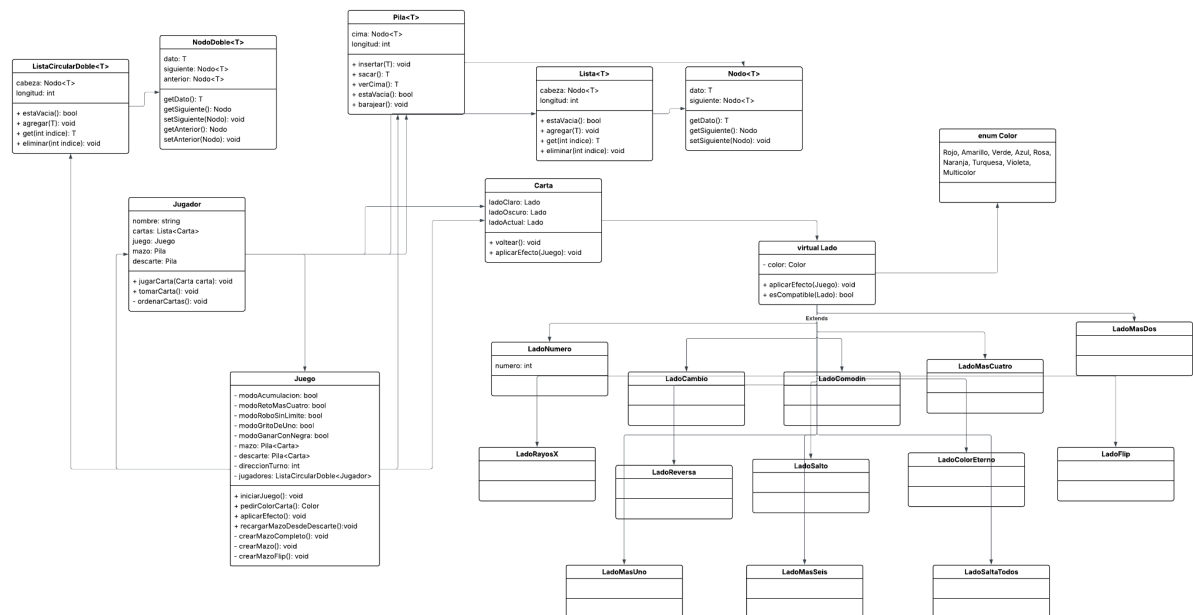
Tecnologías utilizadas

C++/GCC 13.3.0

cmake 3.28.3

vs code 1.105.1

Diagrama de clases



Tipos Abstractos de Datos

TAD Consola

Datos:

- (sin atributos públicos definidos)

Operaciones:

- limpiarPantalla() : void
- esperarContinuar() : void
- leerOpcionMenu(mensaje: string) : int
- pedirTextoNoVacio(mensaje: string) : string
- pedirEnteroMinimo(mensaje: string, minimo: int) : int
- pedirIndiceCarta(totalCartas: int, mensaje: string) : int
- boolTexto(valor: bool) : const char*
- pedirColorClaro(nombreJugador: string) : Color
- pedirColorOscuro(nombreJugador: string) : Color

TAD Carta

Datos:

- *ladoClaro* : Lado
- *ladoOscuro* : Lado
- *ladoActual* : Lado

Operaciones:

- *crearCarta*(claro: Lado, oscuro: Lado) : Carta
- *voltear*(c: Carta) : Carta
- *aplicarEfecto*(c: Carta, j: Juego) : Juego
- *ladoActual*(c: Carta) : Lado

TAD Juego

Datos:

- *jugadores* : ListaCircularDoble<Jugador*>
- *mazo* : Pila<Carta*>
- *descarte* : Pila<Carta*>
- *consola* : Consola
- *validador* : Validador
- *direccionTurno* : int
- *saltosPendientes* : int
- *cartasRoboPendientes* : int
- *retoMasCuatroPendiente* : bool
- *jugadorMasCuatro* : Jugador*
- *colorAnteriorMasCuatro* : Color
- *numeroAnteriorMasCuatro* : int
- *numeroAnteriorValido* : bool
- *jugadorPendienteUno* : Jugador*
- *jugadorActual* : Jugador*
- *nodoActualTurno* : NodoCircularDoble<Jugador*>*
- *modoAcumulacion* : bool
- *modoRetoMasCuatro* : bool
- *modoRoboSinLimite* : bool
- *modoGritoDeUno* : bool
- *modoGanarConNegra* : bool
- *modoFlip* : bool
- *colorEternoPendiente* : bool
- *colorEternoObjetivo* : Color
- *repetirTurnoActual* : bool

Operaciones:

- *iniciarJuego*() : void
- *pedirColorCarta*(nombreJugador: string) : Color
- *aplicarReversa*() : void
- *aplicarSalto*() : void
- *aplicarMasDos*() : void
- *aplicarMasCuatro*() : void
- *aplicarMasUnoFlip*() : void
- *aplicarMasDosFlip*() : void
- *aplicarMasTresFlip*() : void
- *aplicarMasSeisFlip*() : void
- *aplicarSaltaTodosFlip*() : void
- *aplicarFlip*() : void

- aplicarColorEterno(colorElegido: Color) : void
- registrarMasCuatro(jugador: Jugador*, topeAntes: Lado) : void
- isModoRoboSinLimite() : bool
- revelarCartasSiguiente() : void
- intercambiarCartasConAnterior() : void
- getNombreJugadorActual() : string
- pedirColorOscuro(nombreJugador: string) : Color
- recargarMazoDesdeDescarte() : void

TAD Jugador

Datos:

- nombre : string
- cartas : Lista<Carta*>
- juego : Juego*
- mazo : Pila<Carta*>*
- descarte : Pila<Carta*>*

Operaciones:

- crearJugador(nombre: string, juego: Juego*, mazo: Pila<Carta*>*, descarte: Pila<Carta*>*) :

Jugador

- tomarCarta(carta: Carta*) : void
- mostrarCartas() : void
- getCantidadCartas() : int
- getNombre() : string
- setNombre(nuevoNombre: string) : void
- getCartas() : Lista<Carta*>&
- tieneCartaCompatible(tope: Lado) : bool
- jugarCarta(indice: int, tope: Lado) : bool
- manejarSinCompatibles(tope: Lado) : void
- robarCartas(cantidad: int) : void
- tieneCartaDeColor(color: Color) : bool
- tieneCartaNumero(numero: int) : bool
- tieneCartaAcumulable(tope: Lado) : bool
- intercambiarCartasCon(otro: Jugador) : void
- voltearCartas() : void
- extraerCarta(indice: int) : Carta*

TAD Validador

Datos:

- (sin atributos)

Operaciones:

- validarJugadoresParaRepartir(totalJugadores: int) : bool
- validarInicioTurnos(totalJugadores: int, jugadorInicial: Jugador*) : bool
- validarJugadorConCartas(jugador: Jugador*, mensaje: string) : bool
- validarMazoNoVacio(mazo: Pila<Carta*>, mensaje: string) : bool

TAD Color

Datos:

- valores = {Rojo, Verde, Azul, Amarillo, Rosa, Turquesa, Naranja, Violeta, Multicolor}

Operaciones:

- colorToString(color: Color) : const char*

TAD Pila<T>

Datos:

- cima : $\text{Nodo}<T>^*$
- longitud : int

Operaciones:

- crearPila() : $\text{Pila}<T>$
- insertar(valor: T) : void
- sacar() : T
- verCima() : T
- isEmpty() : bool
- getLongitud() : int
- barajear() : void
- algoritmoBarajeado(arreglo: T^* , n : int) : void

TAD Lista<T>

Datos:

- cabeza : $\text{Nodo}<T>^*$
- longitud : int

Operaciones:

- crearLista() : $\text{Lista}<T>$
- estaVacia() : bool
- getLongitud() : int
- agregar(dato: T) : void
- get(indice: int) : T
- eliminar(indice: int) : void
- intercambiar(otra: $\text{Lista}<T>$) : void

TAD Nodo<T>

Datos:

- dato : T
- siguiente : $\text{Nodo}<T>^*$

Operaciones:

- crearNodo(dato: T) : $\text{Nodo}<T>$
- getDato() : T
- getSiguiente() : $\text{Nodo}<T>^*$
- setSiguiente(siguiente: $\text{Nodo}<T>^*$) : void

TAD ListaCircularDoble<T>

Datos:

- cabeza : $\text{NodoCircularDoble}<T>^*$
- longitud : int

Operaciones:

- crearListaCircularDoble() : $\text{ListaCircularDoble}<T>$
- insertar(valor: T) : void
- eliminar(indice: int) : void
- get(indice: int) : T
- buscar(valor: T) : bool
- getLongitud() : int
- isEmpty() : bool
- imprimir() : void
- getCabeza() : $\text{NodoCircularDoble}<T>^*$
- getNode(indice: int) : $\text{NodoCircularDoble}<T>^*$

TAD NodoCircularDoble<T>

Datos:

- dato : T
- siguiente : NodoCircularDoble<T>*
- anterior : NodoCircularDoble<T>*

Operaciones:

- crearNodoCircularDoble(valor: T) : NodoCircularDoble<T>
- getDato() : T
- getSiguiente() : NodoCircularDoble<T>*
- getAnterior() : NodoCircularDoble<T>*
- setDato(valor: T) : void
- setSiguiente(nodo: NodoCircularDoble<T>*) : void
- setAnterior(nodo: NodoCircularDoble<T>*) : void

TAD Lado (abstracto)

Datos:

- color : Color

Operaciones:

- aplicarEfecto(juego: Juego) : void
- esCompatible(otro: Lado) : bool
- toString() : string
- getColor() : Color
- setColor(nuevoColor: Color) : void

TAD LadoNumero

Datos:

- numero : int

Operaciones:

- crearLadoNumero(num: int, color: Color) : LadoNumero
- aplicarEfecto(juego: Juego) : void
- esCompatible(otro: Lado) : bool
- toString() : string
- getNumero() : int

TAD LadoCambio

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoCambio(color: Color) : LadoCambio
- aplicarEfecto(juego: Juego) : void
- esCompatible(otro: Lado) : bool
- toString() : string

TAD LadoComodin

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoComodin() : LadoComodin
- aplicarEfecto(juego: Juego) : void
- esCompatible(otro: Lado) : bool
- toString() : string

TAD LadoMasDos

Datos:

- color : Color

Operaciones:

- crearLadoMasDos(color: Color) : LadoMasDos

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoMasCuatro

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoMasCuatro() : LadoMasCuatro

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoReversa

Datos:

- color : Color

Operaciones:

- crearLadoReversa(color: Color) : LadoReversa

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoSalto

Datos:

- color : Color

Operaciones:

- crearLadoSalto(color: Color) : LadoSalto

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoRayosX

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoRayosX(color: Color) : LadoRayosX

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoFlip

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoFlip(color: Color) : LadoFlip

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoMasUnoFlip

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoMasUnoFlip(color: Color) : LadoMasUnoFlip

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoMasDosFlip

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoMasDosFlip(color: Color) : LadoMasDosFlip

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoMasTresFlip

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoMasTresFlip(color: Color) : LadoMasTresFlip

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoMasSeisFlip

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoMasSeisFlip(color: Color) : LadoMasSeisFlip

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoSaltaTodos

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoSaltaTodos(color: Color) : LadoSaltaTodos

- aplicarEfecto(juego: Juego) : void

- esCompatible(otro: Lado) : bool

- toString() : string

TAD LadoColorEterno

Datos:

- (sin atributos propios)

Operaciones:

- crearLadoColorEterno() : LadoColorEterno

- *aplicarEfecto(juego: Juego) : void*
- *esCompatible(otro: Lado) : bool*
- *toString() : string*

Análisis de complejidad

Barajar en Pila

Se copian todos los nodos a un arreglo, se barajea con un ciclo simple y se reinserta en la pila, cada paso tiene una complejidad lineal.
La complejidad es lineal $O(n)$.

Buscar una carta en la mano del jugador

La mano del jugador es una lista enlazada, y en cada búsqueda se recorre nodo por nodo linealmente.
La complejidad es lineal $O(n)$.

Compilación del proyecto

Linux:

Ejecutar: `make`, en la carpeta `/build`

O ejecutar en la raíz del proyecto:

```
cmake -S . -B build-linux  
cmake --build build-linux
```